

ARP Cache Poisoning and Man-in-the-Middle Attack

Computer Security Sessional

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology (BUET)

Final Report & Implementation Demo

13th–14th Week

Submitted by

2105095 — Farihan Israaq
2105108 — Showkat Nur Daiyan

18 September 2026

1. Overview and experimental setup

This project demonstrates ARP cache poisoning and a man-in-the-middle attack inside an isolated Docker network. Forged ARP replies redirect traffic between a victim and a local server through an attacker. The implementation can observe plaintext traffic, modify HTTP responses, selectively block HTTP, and delay forwarding. An endpoint ARP inspection defense is implemented and evaluated against the same attack.

Result: Saved experiments show successful interception and modification without protection. With ARP inspection enabled on both endpoints, the attacker intercepted no targeted IP packets, while the integration checks required normal HTTP and DNS communication.

Device / role	IPv4 / MAC address	Function
Victim / client	10.0.0.10 02:42:0a:00:00:0a	HTTP requests and DNS queries
Gateway / server	10.0.0.1 02:42:0a:00:00:01	HTTP on TCP 8080; DNS on UDP 53
Attacker	10.0.0.66 02:42:0a:00:00:42	ARP poisoning and Ethernet relay

NORMAL PATH

Victim (10.0.0.10) <-----> Server (10.0.0.1)

POISONED PATH

Victim <-----> Attacker (10.0.0.66) <-----> Server
observe / modify / drop

All three PCs are represented by Linux containers on an internal 10.0.0.0/24 Docker bridge. The Docker bridge gateway is 10.0.0.254. The container named gateway is a demonstration server, not an Internet router. No ports are published to the host. Python handles raw Ethernet forwarding; kernel IPv4 forwarding remains disabled.

Evidence basis. This report uses the current source code and saved integration runs from 18 September 2026, approximately 11:42–11:43 Asia/Dhaka. Totals are taken from integration-results.json. Selected event and statistics files and the integration test were inspected during preparation; those per-run test folders and tests/integration.py were subsequently removed from the workspace. Experiments were not rerun for this report because the Docker daemon was unavailable. Original desktop screenshots and raw victim terminal transcripts were not found; the labeled panels on page 4 distinguish saved evidence from reconstructed screen output.

2. Attack procedure and implementation

Step 1 — Start the lab and establish a baseline. Start Docker Desktop, then use the commands below. The victim should receive ORIGINAL: Hello from the lab gateway! and resolve demo.lab to 10.0.0.1. Disable an existing defense before demonstrating an unprotected attack.

```
docker compose up --build -d --wait
python3 -m arplab.inventory disable
docker compose exec victim python -m arplab.client traffic
```

Step 2 — Discover and poison both peers. The attacker discovers the real peer MAC addresses, then repeatedly sends forged ARP replies. To the victim it claims that 10.0.0.1 belongs to 02:42:0a:00:00:42; to the server it claims that 10.0.0.10 belongs to that same attacker MAC. Both endpoints then direct peer traffic through the attacker.

Step 3 — Start HTTP modification in terminal A. Wait for the ready event. The relay preserves the IP endpoints and forwards frames using the genuine destination MAC. Default modification replaces ORIGINAL with MODIFIED and repairs checksums.

```
docker compose exec attacker python -m arplab.attack \
  --mode modify --duration 60 --output /artifacts/report-modify
```

Step 4 — Generate victim traffic in terminal B. Observe the changed HTTP body, successful DNS, and both poisoned neighbor entries.

```
docker compose exec victim python -m arplab.client traffic
docker compose exec victim ip neigh show
docker compose exec gateway ip neigh show
docker compose exec victim python -m arplab.client alerts
```

Step 5 — Compare other attack modes. Repeat with --mode relay for observation without modification; --mode drop for HTTP blocking while DNS is forwarded; or --mode relay --delay-ms 100 for a configured delay per forwarded frame. Use a distinct output folder for each run.

Step 6 — Restore the network. Normal completion and SIGINT/SIGTERM trigger genuine ARP restoration attempts. Verify the original peer MAC addresses and a successful ORIGINAL response afterward. Forced termination may leave stale mappings. Stop the lab with docker compose down after the demonstration.

Menu alternative: python3 main.py; choose 1 to start, 2 for baseline, 3 for relay, 4 for custom HTTP text, 5 for blocking, 6 for defense, 7 to disable defense, 8 to inspect state, and 10 to stop. Custom text is padded to the 1024-byte demo body; it must fit within 1023 UTF-8 bytes.

3. Evidence snapshots and victim observations

Snapshot 1 — Attacker log excerpt (saved evidence). Selected fields below come from the modification integration run. Long padding and unrelated packets are omitted; the original body was recorded before modification.

```
ready: mode=modify
request: GET / HTTP/1.1; Host: 10.0.0.1:8080
server body: ORIGINAL: Hello from the lab gateway!
modified: original=ORIGINAL, replacement=MODIFIED
summary: received=13, forwarded=13, modified=1, dropped=0
```

Source: artifacts/test-modify-1789710160362393000/events.jsonl.

Snapshot 2 — Victim terminal (reconstructed view). This panel presents the client output format for the outcome checked by the integration script; it is not an original captured terminal screenshot.

```
$ docker compose exec victim python -m arplab.client traffic
{"action": "http",
 "result": "MODIFIED: Hello from the lab gateway!"}
{"action": "dns", "result": "10.0.0.1"}
```

Snapshot 3 — Neighbor mappings (test-checked reconstruction). The integration test checks the attacker MAC in both unprotected peer entries. Exact neighbor states and complete terminal lines were not saved.

```
Victim entry for server:
10.0.0.1 lladdr 02:42:0a:00:00:42
Server entry for victim:
10.0.0.10 lladdr 02:42:0a:00:00:42
```

Snapshot 4 — Protected attacker statistics (saved evidence). The protected modification run attempted nine poisoning rounds but received no targeted IP packets.

```
poison_rounds=9
received=0    forwarded=0    modified=0    dropped=0
victim_to_gateway=0    gateway_to_victim=0
```

Source: artifacts/test-defense-1789710195206302000/stats.json. The associated integration checks require ORIGINAL HTTP, successful DNS, genuine dynamic neighbor entries, and positive rejected-ARP counters at both endpoints. Exact counter values were not saved in this results file.

4. Attack results and success analysis

The attack succeeded in the unprotected configuration. Bidirectional interception, a changed HTTP response, and selective packet dropping demonstrate confidentiality, integrity, and availability impacts. The saved scenario totals are shown below.

Scenario	Received	Forwarded	Dropped	Modified	Poison rounds
Relay	13	13	0	0	9
Modify HTTP	13	13	0	1	9
Drop HTTP	5	2	3	0	9
Delay 100 ms	13	13	0	0	9
Protected modify	0	0	0	0	9

Confidentiality: The attacker log exposes the HTTP request, original response body, and DNS name demo.lab. The relay forwarded 13 selected packets: seven from victim to server and six in the reverse direction. Normal service continued through the attacker.

Integrity: The modification run records one changed response packet and 13 forwarded packets. The integration script requires the victim HTTP result to contain MODIFIED before recording that scenario. This connects the attacker action to application-level acceptance rather than relying only on a modification counter.

Availability: The blocking run records three dropped HTTP packets and two forwarded DNS packets. Its test requires an HTTP error and a successful DNS response. This supports selective HTTP disruption, rather than a total network outage.

Factors enabling the attack. Dynamic ARP mappings accepted forged claims for the peer IP addresses. Poisoning both directions placed the relay in the communication path. Plaintext HTTP allowed content replacement. Equal-length replacement preserved TCP sequence numbering; checksum completion supported raw forwarding of packets whose checksums might otherwise depend on interface offloading.

Effectiveness of the protected configuration. Endpoint ARP filters rejected inconsistent sender bindings before neighbor-cache updates. Nine poisoning rounds intercepted zero targeted packets. The saved test workflow also required normal HTTP/DNS and genuine dynamic peer entries, providing evidence that protection preserved legitimate communication.

Packet counts describe these specific runs. The delay experiment forwarded 13 packets with a configured 100 ms delay per frame; no measured end-to-end latency result was retained.

5. Observations across the lab devices

Scenario	Attacker PC	Victim PC	Server / gateway PC
Baseline	No attack needed.	ORIGINAL response; DNS: 10.0.0.1.	Generates the original HTTP body and DNS answer.
Relay	13 received and forwarded; HTTP and DNS details visible.	ORIGINAL response; DNS succeeds.	Six return packets seen by attacker; peer mapping poisoned.
Modify	13 forwarded; one modified response packet.	MODIFIED response; DNS succeeds.	Original body appears in the incoming attacker log before alteration.
Drop	Five received; three HTTP packets dropped; two DNS packets forwarded.	HTTP error; DNS succeeds.	DNS reply traverses the relay; intercepted HTTP attempts are dropped.
Delay	13 received and forwarded with configured delay.	ORIGINAL response; DNS succeeds.	Bidirectional traffic continues.
Protected modify	Nine poison rounds; zero intercepted or modified packets.	ORIGINAL response; genuine dynamic server mapping.	Genuine dynamic victim mapping; DNS remains available.

Evidence supporting the observations. Attacker counts and payloads come from saved per-run JSON files. Victim outcomes and neighbor mappings are supported by the saved integration results and assertions in tests/integration.py; raw terminal transcripts were not retained. Server behavior is supported by the original response captured in the attacker log, the service implementation, and the checked peer mappings.

Detection output. The endpoint watcher writes arp_alert events. Its implemented reasons include trusted_mapping_mismatch, unsolicited_mapping_change, and one_mac_claims_multiple_ips. After trusted inventory enrollment, a claim that pairs the server IP with the attacker MAC can be compared against that inventory. Alerts can still appear while filtering blocks the attempt; an alert is evidence of detection, not proof that poisoning succeeded.

Other related PCs. There is no fourth remote client or Internet server in this experiment. The host computer controls the three containers. The gateway container provides both application services. HTTP access logging is disabled in the server implementation, so no separate server access-log transcript is claimed.

6. Countermeasure design and evaluation

The implemented countermeasure uses managed-inventory ARP inspection. It operates on both the victim and server, retaining normal dynamic ARP resolution while filtering forged sender claims.

1. Obtain a trusted inventory. arplab/inventory.py reads the running containers' IP/MAC assignments through Docker inspection on the host. It verifies the three roles and their shared network, then sends bindings to each endpoint through docker compose exec. Bindings are stored in /run/arplab/bindings.json. Trust does not come from a possibly poisoned ARP cache.

2. Filter ARP before cache updates. arplab/defense.py installs an nftables ARP input chain on eth0. A packet is accepted only if the ARP sender IP, ARP sender MAC, and Ethernet source MAC all match an enrolled binding. A final rule counts and drops other ARP packets. Both ARP requests and replies are checked.

Accept only when all three fields match an enrolled host:

```
ARP sender IP      = enrolled IP
ARP sender MAC     = enrolled MAC
Ethernet source MAC = enrolled MAC
```

Otherwise: count and drop the ARP packet.

3. Clear stale peer state and verify. Enabling or refreshing the rules flushes the peer neighbor entry so genuine ARP resolution can occur again. Neighbor entries remain dynamic; this version does not pin permanent entries or rely on ARP sysctl changes.

```
python3 -m arplab.inventory enable
# Repeat the modification attack, then inspect:
python3 -m arplab.inventory status
docker compose exec victim nft list table arp arplab_guard
docker compose exec victim ip neigh show
docker compose exec gateway ip neigh show
```

Evaluation. The saved protected run reports zero received and modified packets after nine poisoning rounds. The integration test requires both filters to be enabled with dropped_arp greater than zero, genuine non-permanent neighbor entries, an ORIGINAL response, and successful DNS. The test records no exact ARP-drop total in integration-results.json.

Limitations. Inventory must be refreshed after legitimate address changes; new hosts require enrollment. An attacker cloning a valid IP and both MAC fields can pass this endpoint check because switch-port identity is not verified. There is no DHCP snooping or automatic lease management. Updates are atomic per endpoint but sequential across endpoints. ARP probes using sender IP 0.0.0.0 are rejected in this fixed-address lab. The passive watcher detects attempts but does not itself block packets.

Conclusion

The saved evidence supports successful ARP-based interception, HTTP response modification, and selective HTTP blocking in the unprotected lab. The implemented ARP inspection defense prevented the tested poisoning technique while preserving normal endpoint communication.